

AI Data Processing & Integrity Controls

Chapter 04 · Episode 01 · Domain 1.2 — Data Integrity Is Model Integrity



Learning Objectives

1. Explain all seven data-processing controls and their function
2. Connect data-quality failures to model security outcomes
3. Apply data integrity controls to a training-pipeline scenario

Why Data Security IS Model Security

TRADITIONAL APP SECURITY

Vulnerability lives in code

Fix the code → patch

Auditable, point-in-time

Surface: running app

AI DATA-ORIGIN

Vulnerability in the weights

Fix requires full retraining

Opaque; found in production

Surface: the data pipeline

Seven Data-Processing Security Controls

- 1 → Cleansing
 - 2 → Verification
 - 3 → Lineage
 - 4 → Integrity
 - 5 → Provenance
 - 6 → Augmentation
 - 7 → Balancing
-



Controls 1–4: Foundation of Trust



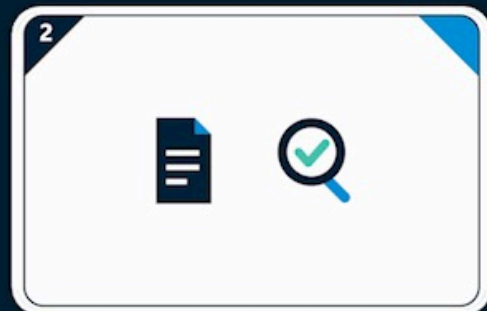
CLEANSING

Remove injected/malformed
(valid $\neq$ safe)



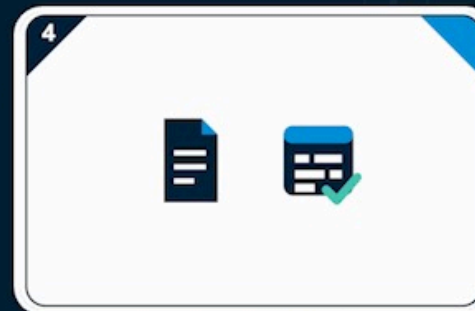
LINEAGE

Source-to-training audit trail



VERIFICATION

Confirm records are authentic



INTEGRITY

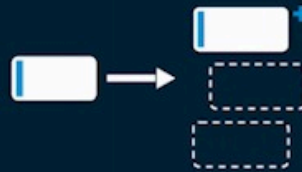
Hashes detect modification

Controls 5–7: Source, Expand, Balance



PROVENANCE

Source legitimacy & consent
EU AI Act / GDPR requirement



AUGMENTATION

Expand data artificially
Review for distortion & bias



BALANCING

Proportional classes
Imbalance = blind spots

1 Cleansing Check

PASS

2 Verification Check

PASS

3 Lineage Check

WARN

4 Integrity Hash

FAIL

5 Provenance Check

PASS

6 Augmentation Review

PASS

7 Balance Analysis

WARN

HOW TO RUN

WINDOWS

```
python data_integrity_pipeline_checker.py
```

MACOS/LINUX

```
python3 data_integrity_pipeline_checker.py
```

TRY IT Change CHECKS[3]['passed'] = True to fix the hash failure. Observe trust score rise and pipeline complete.

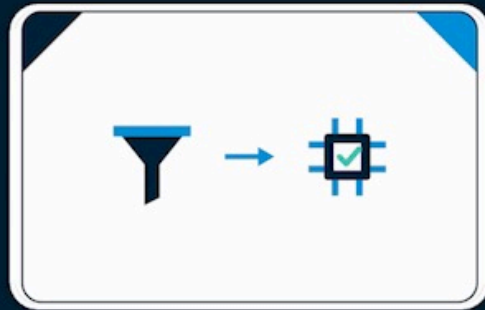
`data_integrity_pipeline_checker.py`

pure Python 3.10+, no external libraries

Demo: Data Integrity Pipeline Checker

Seven checks in sequence — an integrity-hash failure blocks the pipeline (trust score insufficient)

Control → Security Outcome



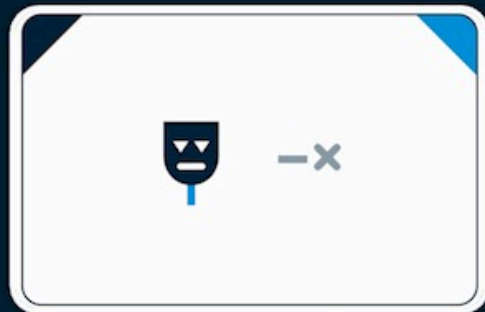
CLEANSING

Prevents garbage models



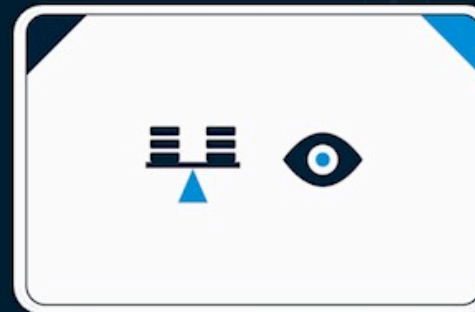
INTEGRITY

Detects tampering



VERIFICATION

No false ground truth



BALANCING

No systematic blind spots

Exam Match Rule

Scenario → which control applies

Missed attack type → balancing / verification

Modified dataset → integrity / lineage

Unauthorized source → provenance

SYMPTOM · STAGE · CONTROL

SOURCE Not authorized for this use

PROVENANCE

TRANSIT Modified after collection

INTEGRITY

LINEAGE

TRAINING SET Misses one attack class

CLEANSING

VERIFICATION

AUGMENTATION

BALANCING

Data Types, RAG, Embeddings & Watermarking

Chapter 04 · Episode 02 · Domain 1.2 — Retrieval & Provenance



Learning Objectives

1. Classify structured, semi-structured & unstructured data + risks
2. Trace a RAG query end-to-end, security concern at each step
3. Explain embeddings, vector stores & watermarking

Three AI Data Types



STRUCTURED

Rows/columns, DBs, logs
PII memorization risk



SEMI

JSON/XML, telemetry
Schema fools cleansing



UNSTRUCTURED

Text/images/audio
Highest — extractable

RAG — Retrieval-Augmented Generation

Augments responses with retrieved knowledge at query time

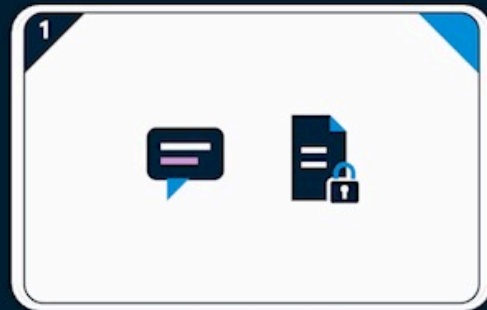
Update the knowledge base without retraining

Keeps sensitive data in a DB, not the weights

Query → embedding → retrieve → augment
→ generate

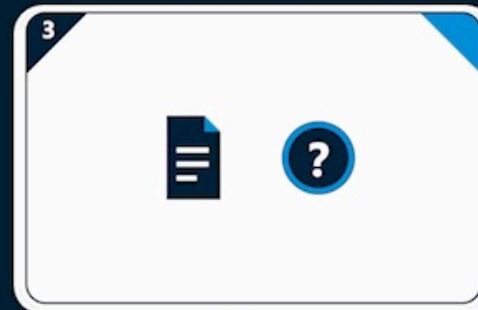


RAG's Attack Surfaces



QUERY

Crafted query targets sensitive docs



AUGMENTATION

Retrieved content unlabeled/untrusted



RETRIEVAL

RAG poisoning of the knowledge base



GENERATION

Leaks retrieved sensitive content

Embeddings & Vector Stores

Vectors encode semantic meaning

'malware' sits near 'ransomware', far from 'sandwich'

Vector stores index them for similarity search

Access control matters as much as the source docs



Watermarking — Two Provenance Applications

TRAINING-DATA WATERMARK

- Markers embedded pre-training
- Model reproduces them in output
- Proves it trained on your data
- Detects unauthorized dataset use

OUTPUT WATERMARK

- Imperceptible signal in output
- Text, images, audio
- Flags content as AI-generated
- Traces it to the model

1 Query → Embedding

PASS

2 Vector DB Retrieval

PASS

3 Content Validation

WARN

4 Prompt Augmentation

FAIL

5 Output Filtering

PASS

HOW TO RUN

WINDOWS

```
python rag_pipeline_security_walkthrough.py
```

MACOS/LINUX

```
python3 rag_pipeline_security_walkthrough.py
```

TRY IT Enable RAG_POISONING_DEMO=True in the config to run a simulated poisoning attack and see how content validation catches it.

`rag_pipeline_security_walkthrough.py`

pure Python 3.10+, no external libraries

Demo: RAG Pipeline Security Walkthrough

Query → embedding → retrieval → augmentation → generation, with the control at each step

Security Reference Map

Structured, semi-structured, unstructured — know the difference

RAG: five steps, an attack surface at each one

Embeddings encode semantic meaning; vector stores need access control

Watermarking is a provenance control, not access control

RAG poisoning: knowledge-base access control and content validation, not prompt templates alone

SECURITY REFERENCE MAP

UNSTRUCTURED
DATA



CLASSIFY
AND REDACT

RAG POISONING



KNOWLEDGE
BASE INTEGRITY

VECTOR STORE



MIRROR
REPO PERMISSIONS

WATERMARKING



PROVENANCE,
NOT ACCESS



FOR THE EXAM

Match the control to the stage it protects — the risk names the answer.

Exam Tips



DATA TYPES

Unstructured = highest risk



VECTOR STORES

Need access control



RAG POISONING

Indirect injection via the KB



WATERMARKING

A provenance control